Table of Contents

NEXUS — Employee Portal & Chassis Leasing Platform

Production architecture, feature inventory, authorization and implementation roadmap

Document type: Living project blueprint
Version: 1.0 — Planning baseline
Date: 12 October 2026
Tech stack: React + Express/Node.js + PostgreSQL (Supabase) + Supabase Auth
Status: Design decisions proposed, not yet implemented

> **How to resume this project after a break:** Start at Section 1 (decisions), check Section 16 (work board), continue the first unfinished milestone, and record decisions in Section 17. Do not code new modules until their permission rules, data owner and acceptance tests are defined.

---

1. Executive decisions and architecture at a glance

Nexus is ONE modular internal platform, not five separately maintained apps. Start with five **base roles**: **Employee**, **Manager**, **HR Administrator**, **Support Agent** and **System Administrator**. All five are staff members. Elevated roles retain normal employee features; permissions are additive, but data visibility remains scoped. A person can hold more than one role. Do not assume that a system administrator can read confidential HR or commercial information.

Decisions to lock now

1. Base access is **RBAC + resource ownership + organization scope + workflow state**, not a hardcoded role check in React.
2. Authenticate with Supabase Auth. Model staff records, roles, permissions, organization structure and approvals in app tables.
3. Treat HR, task management, support and commercial leasing as separate domains with clear boundaries.
4. Managers authorize their team's ordinary leave requests. HR configures policies, audits and corrects balances; HR is an exception/alternate approver when explicitly assigned, not an automatic bypass.
5. Support agents own queues. HR issues go to HR-authorized agents; IT tickets go to IT agents; payroll cases to authorized payroll agents. A generic helpdesk agent cannot read every queue.

6. Maintain one immutable or append-only record of who requested, reviewed, approved, rejected, reassigned or corrected each important item.
7. Use company-specific calendars, employment dates and business-timezone rules; timestamps stored in UTC with the originating timezone preserved where relevant.
8. Chassis sales/leasing is an optional **business module** granted through capabilities such as sales.opportunity.read and leasing.contract.manage; do not introduce a sixth universal role just to give a sales employee access.
9. Default to least privilege, no public staff signups, managed invitations and verified account activation.
10. Leave complex payroll, GPS-based time tracking, AI ticket actions and accounting integrations for later phases.

Working assumptions: Single leasing company at first; one employee may report to one primary manager; HR controls organizational data; business operations may manage equipment and customers; sales/lease rules are placeholders pending business clarification. The schema should allow multi-entity business units and multiple reporting assignments later without forcing multi-tenancy now.

2. Personas / role definitions

2.1 Employee

Purpose: Self-service and collaboration.
Can access: Dashboard, own profile, own attendance and leaves, assigned/created tasks, staff directory with approved public-work fields, company calendar, policies, own helpdesk tickets and authorized org chart.
Cannot access: Colleague's private contact information, salary, performance details, absence reasons, protected ticket threads, other people's time records or security settings.

2.2 Manager

Purpose: Team operations and delegated approvals.
Everything an Employee can do, plus: Team dashboard, direct-report roster, team attendance exceptions, leave approval queue, team leave calendar, work allocation, task assignment/reporting and escalations.
Boundaries: Direct reports or explicitly delegated subteams; no automatic access to medical attachments, salary, HR investigations or unrelated departments. A manager's own leave must go to their manager or a configured alternate.

2.3 HR Administrator

Purpose: People operations and records management.
Everything an Employee can do, plus: Employee lifecycle records, hiring/onboarding/offboarding workflow, leave policy and balances, company holidays, policy publishing/acknowledgments, department/job data, HR reporting and restricted HR service queues.

Boundaries: Scoped authorized HR users only; HR visibility does not automatically cover all IT/private support ticket details. Changes to balances and overrides require rationale and audit entries.

2.4 Support Agent

Purpose: Fulfill assigned service requests.
Everything an Employee can do, plus: Queue inbox, ticket assignment, status and priority updates, internal notes, SLAs, knowledge-base contributions (subject to approval), escalation and ticket reporting for authorized queues.
Boundaries: Category-based grants (IT, HR, payroll, facilities); private messages/attachments must be filtered per queue and case membership. Can be combined with HR Admin for an HR employee, but should not assume it.

2.5 System Administrator

Purpose: Technical and security operations.
Everything an Employee can do, plus: Invitation and account management, role and permission grants, feature flags, organization/system configuration, integration settings, audit/security logs, environment health and incident response.
Boundaries: Does not read sensitive HR case contents or customer prices/contracts automatically. Exceptional access requires a break-glass procedure, time limit, reason and audit review.

Role stacking example: Morgan is employee + manager + sales permissions. Taylor is employee + HR administrator + support agent (HR queue only). Store base roles in a join table; domain grants may be attached to roles, users or groups.

3. Existing employee pages: keep / improve / defer

Page	Now / keep	Add or improve in the first real release	Defer
Dashboard	Clock in/out, quick links, highlights, holidays, birthdays, anniversaries	Current attendance state; compact leave balance; pending approvals/tasks/tickets; must-read policy banners; pinned shortcuts with recent pages optional	Mood question (unless voluntary, privacy-governed and actionable); personalization engine

Page	Now / keep	Add or improve in the first real release	Defer
Task Box	Create/assign task, title, issue type, priority, description, assignee, reporter, labels, due date, attachments	Status and activity history; filters; creator vs assignee; notification preferences; permissions by project/team; clear distinction from helpdesk	Custom workflows, recurring tasks, automations, complex dependencies
Profile	Personal and job overview	Split public work info from private HR records; employee-editable fields require verification where appropriate; employment history controlled by HR	Self-service benefits/payroll
Time Management	Clock logs, leave request, balances, metrics, holidays	Attendance events vs daily summary; missed-punch correction workflow; approval queue for manager; leave balance ledger and accrual rules	Biometric integration, payroll-grade attendance, overtime calculation
Employees	Search/filter and staff cards	Click-through modal showing only work-appropriate fields (role, team, manager, start date if policy permits, business email); no automatic display of direct reports' personal details	Employee skills/social profiles
Calendar	Leave, holidays, birthdays and anniversaries	Personal vs team vs all-company views; status legend; organization timezone; filter by event type; request detail permissions	Calendar sync and room bookings
HR Policies	Searchable policy	File viewer with	E-signature vendor

Page	Now / keep	Add or improve in the first real release	Defer
	cards	signed URLs and version history; acknowledgment status; owner, effective date and audience; policy archive	integrations
Helpdesk	Ticket creation, priorities, list and FAQs	Categories, queue ownership, SLA, requester-visible/internal comments, restricted attachments, ticket history	AI answer assistant, external vendors, automated remediation
Org View	Tree/list/card variants	One underlying reporting model; view only authorized work information; identify current manager and direct reports	Large interactive cross-company graph
Notifications	Bell and notification settings	Event subscriptions, mark read, link to target, in-app and email delivery settings	Push/mobile and complicated digest rules
Settings	Preferred date format	Timezone, locale, notification channels; profile security linked to Supabase Auth	Theme customizations

Important: Priority labels must mean something. Tasks can use critical / high / medium / low and support incidents can use urgent / high / normal if those are purposeful; never use visual color as the only encoding. Rename “Task Box — Create Ticket” to **Create Task**; reserve **ticket** for Helpdesk.

4. Role-by-role navigation

Shared navigation (all authenticated staff): Dashboard; My Tasks; My Time; Directory; Calendar; Policies; Helpdesk; Organization; My Profile; Notifications; Preferences.

Manager section (only when permitted): Team Overview; Team Calendar; Leave Approval Inbox; Attendance Exceptions; Team Tasks; Team Reports.

HR Administration section: People Directory (HR); Employee Records; Onboarding/Offboarding; Leave Policies & Adjustments; Holiday Calendar; Policies & Acknowledgments; HR Requests & Reports.

Support Operations section: Queue Inbox; My Assigned Tickets; SLAs & Escalations; FAQ / Knowledge Base; Queue Analytics.

System Administration section: Invitations & Accounts; Roles & Permissions; Departments/Org Configuration; Feature Flags; Integrations; Audit & Security Logs; System Health.

Sales & Leasing module (feature/permission gated): Sales Dashboard; Customers; Leads & Opportunities; Chassis Inventory; Quotes; Contracts; Reservations/Allocations; Returns; Reports. This is a business capability, not one of the five required base roles.

UX rule: Show only authorized navigation items; redirect on forbidden routes; APIs and RLS MUST independently enforce permissions. UI hiding is not security.

5. Capability matrix — first production target

Legend: **O** own, **T** team/manager-assigned scope, **H** authorized HR staff scope, **Q** assigned queue, **C** configuration, **V** organization-safe public-work fields, **—** not granted by default. These are default grants, not proof of actual row access; every operation also checks ownership, workflow state and privacy.

Capability / action	Employee	Manager	HR Admin	Support Agent	System Admin
profile.read_public	V	V	V	V	V
profile.update_self	O	O	O	O	O
employee.read_private	—	—	H	—	—
employee.update_employment	—	—	H	—	—
attendance.clock_self	O	O	O	O	O
attendance.read	O	T	H	O	O
attendance.c	Submit request	Review T	H	Submit request	—

Capability / action	Employee	Manager	HR Admin	Support Agent	System Admin
orrect					
leave.request	O	O	O	O	O
leave.read	O	T	H	O	O
leave.approve	—	T	Exception-only H	—	—
leave.policy.manage	—	—	H	—	—
task.create	Assigned scope	Team	Assigned scope	Assigned scope	Assigned scope
task.assign	If permitted	T	HR scope	Q	—
ticket.create	O	O	O	O	O
ticket.read	O	O	H for HR queue	Q	—
ticket.triage	—	—	HR queue	Q	—
policy.publish	—	—	H	—	—
policy.acknowledge	O	O	O	O	O
org.read	V	V+T	H	V	C (metadata)
account.invite	—	—	Submit request	—	C
role.assign	—	—	—	—	C, restricted
audit.read_security	—	—	—	—	C
sales.* / leasing.*	Grant only	Grant only	Grant only	Grant only	Config only

Explicit deny examples: The requester cannot approve their own leave. A support agent assigned to an IT queue cannot read a payroll ticket. A terminated worker cannot start a new attendance session. A former manager cannot keep access after transfer. A user cannot change manager_id, role_id or employment_status through a normal profile-edit endpoint.

6. Login, onboarding and account lifecycle

Do not create separate login credential stores for each role. Supabase Auth owns identities, password reset, verified email, sessions and MFA where enabled. auth.users.id is

linked to the application's profiles.user_id. Your staff record is separate (employees). Never store passwords in public application tables; never put any admin/service secret in React.

Recommended invitation flow

1. HR creates a staff record with department, job title, start date, reporting manager and status pending_onboarding. No active access yet.
2. HR requests account provision (or is allowed a limited create operation). System administrator or authorized identity workflow creates/invites a Supabase Auth user. Do not allow arbitrary public signup to join the company.
3. Link auth_user_id to the employee in a transactionally safe workflow, or reconcile via a validated invitation token. Store no plain-text invitation token.
4. Grant default Employee role and any approved additional roles/capabilities with effective dates. Assign queue membership or department scope separately.
5. Employee accepts invitation, verifies identity/email as applicable, establishes credentials and completes required onboarding.
6. First sign-in checks employment status, account status and tenant/company membership server-side. Show only authorized pages.
7. Manager/HR transfers trigger immediate re-evaluation of scopes and pending approvals.
8. On offboarding, revoke app eligibility, end active sessions where possible, deactivate or ban login as supported, revoke grants, reassign tickets/tasks, preserve audit records and follow retention rules. JWTs may remain valid until expiration; sensitive endpoints must recheck active account/session status.

Auth fields: user_id, verified work email, status invited/active/suspended/deactivated, invited_at, last_seen_at. Role grants must be server-controlled. Do not rely on client-editable user_metadata for authorization. Do not let the first registrant become admin automatically. Seed admin via a controlled bootstrap script.

Token caveat: JWT custom role claims can become stale when jobs or roles change. Either authorize high-risk actions using current DB grants in the trusted backend/database or use short-lived claims with a clear refresh/revocation strategy. Never rely only on claims for a sensitive approval without rechecking current assignment.

7. Access enforcement model (the most important technical section)

Evaluate every action as (subject, action, resource, organization_scope, workflow_state, time).

Layer 1 — React: Resolve authorized navigation and controls to improve UX; never treat this as enforcement.

Layer 2 — Express API: Verify Supabase JWT server-side, load active account and current grants, validate input (schema validation), check ownership, reporting graph/queue membership, record state and applicable policy, then perform the mutation in a transaction.

Layer 3 — PostgreSQL: Turn on RLS with explicit grants for every exposed table and storage object. For backend-only tables, prefer a non-exposed schema and narrow database privileges. Use RLS for direct Supabase browser access; privileged operations may be routed exclusively through Express with tightly controlled server credentials and SQL transactions. Avoid duplicate, contradictory authorization systems: specify which endpoint path owns each sensitive write.

Layer 4 — Audit: Record actor, action, entity type/id, before/after where appropriate (omit sensitive secrets), reason, request ID and timestamp. Production audit data should be protected from normal application edits.

Permission grant design: roles, permissions, role_permissions, user_roles, user_permission_grants (optional), team_assignments, support_queue_members, feature_flags. Add effective-from/to timestamps and grant-by/grant-reason for sensitive rights. Grants can be unioned but explicit restricted data boundaries still apply.

Example authorization checks:

```
can(user, 'leave.approve', leave):
  user active AND leave.status == 'pending'
  AND leave.employee_id != user.employee_id
  AND user is CURRENT assigned approver OR current authorized delegate
  AND approver grant effective now
  AND request within company scope

can(user, 'ticket.read', ticket):
  user active AND company matches
  AND (ticket.requester_id == user.employee_id
      OR user is authorized assigned agent/queue member
      OR explicit named case participant)
  AND confidential case access rules satisfied

can(user, 'sales.quote.approve', quote):
  user has sales.quote.approve for this branch/company
  AND quote.status == 'submitted'
  AND approval amount within approval limit
  AND user is not disqualified by segregation-of-duties policy
```

Security test grid: For each endpoint/table test unauthenticated, ordinary employee, manager of owner, manager of another department, HR authorized, support agent of right queue, wrong queue, sysadmin without content grant, disabled user, stale role grant and attempt to approve own request. Test reads, creates, edits and deletions, not merely route visibility.

8. PostgreSQL / Supabase data domains (conceptual schema)

Use UUID PKs, created_at/updated_at (timestamptz), optional company_id, audit fields and constraints. This is a conceptual inventory, **not runnable DDL**.

Identity & organization

- companies(id, name, timezone, status) — single row initially, organization-ready.
- profiles(user_id -> auth.users.id, display_name, avatar_path, preferred_timezone, date_format) — public/app identity.
- employees(id, company_id, auth_user_id UNIQUE NULL, employee_no UNIQUE/company, first_name, last_name, work_email, manager_id NULL, department_id, job_title_id, hire_date, employment_status).
- employee_private(id, employee_id, personal_email, personal_phone, date_of_birth, emergency_contact...) — restricted HR schema/table, not part of directory query.
- departments(id, company_id, parent_department_id, name), job_titles, reporting_assignments (if multiple managers / history necessary).
- roles(id, code), permissions(id, code), role_permissions(role_id, permission_id), user_roles(user_id, role_id, scope_type, scope_id, starts_at, ends_at, granted_by).
- team_delegations, support_queues, support_queue_members, feature_flags.

Attendance & leave

- attendance_events(id, employee_id, type IN clock_in/clock_out, occurred_at, source, created_by) — append-only source of truth; multiple shifts supported later.
- attendance_corrections(id, event_id, proposed_time, reason, requester_id, reviewer_id, status); accepted adjustments create auditable corrected events or linked adjustment records.
- attendance_daily_summaries — computed/rebuildable reporting view, not authoritative input.
- leave_types(id, company_id, code, eligibility_rule...), leave_policies (versioned), leave_balances (optional cached projection), leave_ledger(id, employee_id, leave_type_id, amount_days, source, effective_date, request_id).
- leave_requests(id, employee_id, leave_type_id, start_date, end_date, partial_day, reason_category, sensitive_reason_path, status, submitted_at, policy_version_id).
- leave_request_days for partial days, holiday exclusions, cross-year requests or differing schedules.
- approval_steps(id, entity_type, entity_id, step_order, approver_id/delegation_id, status, acted_at, comments); immutable approval_events.
- holiday_calendars, holidays, work_schedules, employee_schedule_assignments.

Tasks & service tickets

- tasks(id, reporter_id, assignee_id, title, description, priority, status, due_at, team_id,...), task_labels, task_comments, task_attachments, task_activity.
- tickets(id, requester_id, assigned_agent_id, queue_id, category_id, priority, status, subject, description, sla_due_at,...), ticket_comments(is_internal), ticket_attachments, ticket_events, ticket_participants.
- knowledge_articles, knowledge_article_versions, faq_categories.
- Separate Task and Helpdesk domains; shared notification and attachment mechanisms are fine.

Policies, events & notifications

- policies(id, category, owner_id, audience_rule, current_version_id), policy_versions(id, policy_id, object_path, effective_at, published_at), policy_acknowledgements(employee_id, version_id, acknowledged_at).
- calendar_events for manual events, with derived feeds for leaves/birthdays/holidays (avoid copying all source data into events).
- notifications(id, recipient_user_id, type, resource_type, resource_id, created_at, read_at), notification_preferences, delivery_attempts.
- files_metadata(id, company_id, owner_id, bucket, object_key, classification, associated_entity_type/id).
- audit_events(id, actor_user_id, action, resource_type, resource_id, request_id, occurred_at, metadata_redacted).

Chassis sales & leasing module

- customers(id, company_id, legal_name, customer_type, status), customer_contacts, leads, opportunities(id, owner_employee_id, customer_id, stage, estimated_value, currency).
- chassis(id, asset_identifier UNIQUE/company, chassis_type_id, serial_vin, purchase_date, asset_status, location_id, owner_entity_id); clarify actual unit identifiers and equipment attributes.
- chassis_types, asset_locations, asset_status_history, inspection_reports, maintenance_records (future).
- quotes(id, customer_id, opportunity_id, status, valid_until, currency, quote_version), quote_lines(id, quote_id, chassis_type_id or chassis_id, quantity, unit_rate, term...), quote_approvals.
- lease_contracts(id, customer_id, quote_id, start_date, end_date, billing_cycle, status, terms_version), contract_lines, allocations(chassis_id, contract_line_id, assigned_from, assigned_until, status), return_inspections.
- sales_orders and sale_items separately if outright equipment sales are supported. Invoices, receipts and tax calculation are later accounting integrations.

Data integrity constraints: Unique active allocation per chassis at overlapping times; enforce lifecycle transitions; no balance below allowed policy without exception; policy

acknowledgment references an exact version; approval references an exact actor and request; use foreign keys, unique checks, indexes on employee/time, queue/status, company_id and contract windows. Protect against concurrent booking/approvals in DB transactions (locking/exclusion constraints as appropriate).

9. Core workflow specifications

A. Leave request (employee → manager → HR controls)

1. Employee views leave balance, applicable holiday calendar and policy details.
2. Employee submits type, date/time range and message; server calculates chargeable units based on schedules, holidays and policy version. Sensitive medical evidence stored separately.
3. Request becomes pending. Create approval step using manager assignment at time of submission; preserve snapshot for historical accuracy while allowing controlled reroute if manager changes.
4. Notify actual approver. Manager sees only requests in their permitted inbox.
5. Manager approves or rejects with optional/required comment. Server verifies current eligibility and request status in one transaction.
6. On approval, append leave ledger debit (or record reservation at submission then settle on approval, but choose exactly one accounting policy); publish team-calendar visibility without exposing private reasons.
7. Notify requester, HR if required and downstream calendar/attendance projection.
8. Cancellation/amendment is a separate auditable transition; refund ledger on approved cancellation. Never erase approvals or ledger history.

Exceptions: Manager unavailable → configured delegate; request by manager → manager's manager; no manager → assigned HR approver; conflict/overlap → reject/hold according to policy; overdrawn balance → policy exception queue. HR override only with explicitly granted authority and reason.

B. Attendance and missed punch

- Clock in → store timestamp and work schedule reference. Clock out → create separate event. Handle repeated clicks and open-shift conflicts idempotently.
- Missed checkout → flag exception; never silently invent checkout time.
- Employee submits correction; manager/HR authorized reviewer accepts/rejects, records old/new values and reason. Aggregates recompute.
- Attendance charts are informational until labor, shift, breaks, overtime and timezone rules are fully implemented.

C. Helpdesk

1. Requester chooses service category (IT, HR, Payroll, Facilities), subject, description, priority and attachments.
2. Category routes to queue; SLA/priority may be normalized by triage policy, not requester alone.

3. Agent with queue membership claims ticket or is assigned; requester sees externally visible responses only.
4. Ticket moves open → triaged → in_progress → waiting_for_requester / waiting_external → resolved → closed, with reopen flow.
5. Agent adds private notes and public replies; attachment permissions follow audience rules.
6. Escalations reassign and notify. HR cases stay HR-restricted; payroll cases have their own confidentiality class.
7. AI (later) retrieves only authorized KB articles and ticket metadata; answers cite sources, disclose uncertainty, do not auto-close or leak employee information.

D. Generic tasks

draft → open → in_progress → blocked → completed / cancelled with assignee, reporter, priority and due date. Multiple assignees can wait. Distinguish internal collaboration task from service-request ticket; cross-link if one triggers the other.

E. Policy acknowledgment

HR uploads/version-controls policy → configures audience and effective date → publishes → employee notification → reads exact version → acknowledges exact version → HR compliance dashboard. Republishing a new version may require reacknowledgment according to policy.

F. Sales & leasing

Lead → opportunity → quotation → approval → signed contract → chassis assignment → active lease → return/inspection → closed. - Sales person creates customers, records lead, updates opportunity and drafts quotation. - Sales manager or specifically granted commercial approver reviews quote (pricing thresholds and conflicts). - Authorized contract operator finalizes lease terms and allocation. - Availability check must avoid double allocation during overlapping dates; quote alone does not automatically reserve a unit unless reservation semantics explicitly say so. - Logistics/yard users may eventually have their own asset-operation capabilities without a sixth base role. - Sale (ownership transfer) and lease (temporary use) are separate finalization flows; implement one business model first.

10. Information classification and date rules

Public company-work fields: Display name, job title, department, business email, manager/office location, work anniversary where company policy allows.
Employee-self fields: Full personal profile, own schedule/clock events, own leave balances and requests, tickets and task permissions.
Manager-scoped: Team availability, high-level absence status, attendance exceptions and leave approval details required for decision. Do not disclose unnecessary medical detail.
HR restricted: Personal addresses, date of birth, employment history, personal contacts, sensitive documents and absence evidence.
Commercial restricted: Customer personal data, quoted rates, discount thresholds, signed

contracts and financial terms.
Security restricted: Token-related metadata, grants, sign-in/audit events, system secrets (secrets never exposed in UI).

Dates: Keep occurred_at as UTC timestamptz; use date for holiday, birthday, policy effective date and leave calendar date; company timezone for shifts and cutoff decisions; per-user preference for display only. Birthday calendar can show month/day without disclosing birth year. Anniversary calculations use hire date without granting full HR records. Holidays may differ by location/employee assignment. Date of birth need not be present in directory API responses.

11. Recommended API/service architecture (PERN + Supabase)

Client: React feature folders (features/attendance, features/leave, features/tasks, features/helpdesk, features/hr, features/leasing, features/admin). Reusable design system, route guards for presentation only and a permission-aware navigation registry.

Express API: auth middleware → validation → authorization → domain service → repository/SQL transaction → audit + outbox → response. Use current verified Supabase identity. Prefer direct Postgres access server-side with tightly scoped credentials and transactions for complex workflows; if using Supabase Data APIs from React, use RLS everywhere, and avoid privileged DB calls from browsers.

Data: Supabase-hosted PostgreSQL, Auth, Storage and optionally Realtime. Use migrations in source control. Configure storage buckets by data classification; signed short-lived URLs and path/metadata checks; never allow arbitrary cross-ticket attachment download.

Notifications: Important transaction records should write an outbox_events entry in the same DB transaction; a worker delivers in-app/email alerts with retries and deduplication. In-app notification unread state is separate from email delivery state.

Feature flag examples: enable_leasing, enable_policy_ack, enable_ai_helpdesk, enable_team_attendance; flags never override permissions.

Suggested folder structure:

```
client/src/
  app/              routing, auth provider, layout, nav registry
  components/       shared UI primitives
  features/         dashboard, tasks, attendance, leave, directory,
                    calendar, policies, helpdesk, org, hr, admin, leasing
  lib/              API client, validation, dates, permissions (UI only)
server/src/
  middleware/       authenticated, authorized, request-validation
  modules/          same feature modules with controllers/services/repositories
  jobs/             outbox processor, scheduled maintenance
```

lib/ db, audit, errors, telemetry
supabase/migrations/ versioned SQL migrations, RLS/grants
supabase/tests/ role-based policy/constraint tests
shared/ API contracts / schemas / domain types
/docs/ architecture decisions, workflow specs, test matrix

12. Production security, privacy and operations baseline

- **Roles are not Supabase's built-in anon / authenticated database roles.** Those are transport identity classes; the five app roles are stored in app data / authoritative claims.
- Enable RLS on exposed tables, restrict SQL grants, test SELECT/INSERT/UPDATE/DELETE for each persona. Handle views with security_invoker where applicable and audit any security-definer function.
- Keep admin/service credentials server-side, outside browser builds, never in source control. Use environment secrets and rotation.
- Invite-only signup, verified email as appropriate, MFA for high-privilege staff, brute-force protections and sensible session policies.
- Apply validation, parameterized SQL, safe attachment content-type/size checks, malware scanning where practical, rate limits and CSRF protections for cookie-based auth.
- Private buckets for HR, tickets, commercial contracts; audit downloads of sensitive files; short-lived signed URLs; document retention and deletion policy.
- Store consent/purpose and country-appropriate privacy policies for birthday displays or any wellbeing survey. Do not implement mood tracking without an explicit, defensible use case.
- Backup/restore drills; staging separate from production; CI migrations + test gate; error reporting, logs with PII redaction, metrics and alerting.
- Use an immutable/restricted audit trail for privileged grants, leave overrides, ticket reassignment and contract approvals.
- Prevent mass assignment in Express: do not accept server-owned fields (approver_id, role_ids, leave_balance, created_by, status) from normal user forms.
- Treat security/HR/commercial access as separate authorization surfaces; make all role-raising mutations auditable and preferably subject to extra verification.

13. Recommended dashboard per role

Employee: Attendance state + clock action; leave summary; upcoming holidays; assigned tasks and own ticket updates; must-read policies; highlights; pinned shortcuts. **Drop mood check** for v1.

Manager: Personal employee widgets plus pending leave approvals, team out today, attendance exceptions, overdue team tasks and direct-report summary; never expose medical reasons on summary cards.

HR Admin: Personal widgets plus onboarding/offboarding due, HR tickets pending, unacknowledged policies, leave policy exceptions and new staff activity.

Support Agent: Personal widgets plus my assigned tickets, unassigned queue tickets, breached / approaching SLA, waiting requester and KB shortcuts.

System Admin: Account invitations pending, disabled/suspicious account alerts, recent permission changes, integration errors, audit exceptions and system health; avoid auto-displaying confidential employee record contents.

Sales capability: Optionally show separate Sales dashboard with pipeline by stage, pending quote approvals, available chassis, expiring leases, contracts awaiting signature and return inspections.

14. Scope roadmap — one finished vertical slice at a time

Phase 0: Decisions and foundation (do first)

Outputs: Domain glossary, role/permission list, org data model, company timezone, Auth and account lifecycle rules, feature navigation registry, migration strategy, test users. **Definition of done:** Can explain who may read/write every planned table and API. No business code yet.

Phase 1: Identity, roles, directory and basic layout

Set up invite-only auth, profiles/employees, roles/grants, departments/managers, route gating, RLS tests, profile and filtered staff directory. **Done:** Employee A cannot read B's private fields; manager transfer changes visibility; suspended user cannot act.

Phase 2: Attendance clock & own record

Clock events, open-shift validation, my attendance history, current status widget, corrections request placeholder. **Done:** Duplicate clock-in and unauthenticated requests rejected; timezone/day boundaries tested.

Phase 3: Leave end-to-end

Leave types/policies, request form, manager approval inbox, ledger balance, team calendar/notifications, cancellation, exception path. **Done:** Only assigned approver can approve; ledger remains consistent under concurrent approvals; employee sees status.

Phase 4: Tasks + Helpdesk

Build separate CRUD/workflows and attachments; add service queues, agent permissions, public/private ticket notes, FAQs. **Done:** Cross-queue access denied and ticket lifecycle audited.

Phase 5: HR core + policies + org

HR employee administration, onboarding/offboarding, policy versions, acknowledgement, org tree and HR-only dashboards. **Done:** Confidential data not exposed to managers or administrators by default.

Phase 6: Sales & leasing first vertical slice

Validate business requirements, customer and chassis catalog, opportunity, quote approval, contract, allocation and closeout. **Done:** Cannot double-book a chassis; non-sales employee cannot read commercial terms; quote workflow has approval audit.

Phase 7: Operational maturity

Reporting, analytics, email worker, proper monitoring, backup restores, penetration review, user training, accessibility/mobile support.

Phase 8: Optional AI + integrations

RAG on vetted accessible KB, ticket suggestion with human-in-the-loop, accounting/CRM integrations, automated billing, maintenance/yard/inspection apps, multi-company support.

Rule: Ship each phase as a testable slice with DB + API + permissions + UI + tests + operational monitoring. Do not build screens disconnected from approved workflows.

15. Leasing-specific business questions to answer before phase 6

1. Does the company lease **container chassis/trailers**, sell chassis outright, or both?
2. Is each unit uniquely serialized (VIN / chassis number / internal asset code)? Are assets fungible by chassis type?
3. Does pricing depend on lease term, region, condition, customer, currency, taxes, deposit, insurance or utilization?
4. Does a quote reserve a physical chassis, or only after contract activation?
5. Who can offer discounts and what are approval thresholds? Who signs contracts?
6. How is pick-up/delivery, handover condition, maintenance, loss/damage and return handled?
7. Are invoices created in Nexus or in external accounting software?
8. Does sales view all customers or only their assigned accounts/branch?
9. Are assets leased across company branches or entities?
10. What is the source of truth for available inventory, contract status and customer credit status?

Until answered, label the module **Sales & Leasing (planned)**, and do not claim that detailed financial workflows are final.

16. Work board — the exact next tasks

- ☐ **P0-01** Confirm company, department and reporting structure; choose primary company timezone.
- ☐ **P0-02** Create glossary: task vs support ticket vs leave request vs approval vs chassis contract.
- ☐ **P0-03** Create seed accounts for all five roles, plus manager from another department and multi-role sales employee.

- ☐ **P0-04** Design employees, employee_private, roles, permissions, user_roles, departments and team_assignments.
- ☐ **P0-05** Write initial permission matrix in code/data and draft allow/deny tests.
- ☐ **P0-06** Implement Supabase invite-only staff provisioning and secure deactivation design.
- ☐ **P0-07** Protect directory and profile tables with proper SQL grants + RLS; write policies/test cases.
- ☐ **P0-08** Add Express JWT verification and permission middleware; document when using Express vs Supabase direct client.
- ☐ **P0-09** Build menu registry based on capabilities; keep common layout, no separate app forks.
- ☐ **P0-10** Validate forbidden API accesses from all role combinations.
- ☐ **P1-01** Finish active/inactive profile and employee directory, with public/private field separation.
- ☐ **P2-01** Attendance events, time boundaries and duplicate handling.
- ☐ **P3-01** Leave request + approver routing + ledger + audit + calendar.
- ☐ **P4-01** Distinguish tasks from helpdesk tickets; implement one full lifecycle each.
- ☐ **P5-01** Versioned policy files/acknowledgments and HR data controls.
- ☐ **P6-01** Conduct chassis workflow discovery before schema/UI implementation.

Weekly restart routine: Open this document → find the first unchecked item → read related domain workflow → run test suite → implement one end-to-end case → update changelog below.

17. Architecture decision log and change register

ID	Decision	Reason	State
ADR-001	One modular application with five base roles	Maintainability and additive capabilities	Proposed
ADR-002	Permission scopes and workflow checks, not simple role pages	Prevent lateral access	Proposed
ADR-003	Supabase Auth owns credentials, public tables own employment data	Separation of identity and business data	Proposed
ADR-004	Express handles sensitive workflow transactions; RLS on any exposed Supabase tables	Consistent cross-layer protection	Proposed

ID	Decision	Reason	State
ADR-005	Manager approves normal leave; HR administers/handles controlled exceptions	Clear segregation of duties	Proposed
ADR-006	Queue-assigned agent handles support; HR tickets restricted	Confidentiality	Proposed
ADR-007	Sales/leasing via optional capabilities, no required sixth base role	Scalable module ownership	Proposed
ADR-008	No mood question in MVP	Privacy complexity and low initial value	Proposed

Session log template:

Date:
Completed:
Open blockers/questions:
Schema/API/UI files changed:
Migration IDs:
Tests added and test result:
New decision (ADR):
Next action (P#):

18. Acceptance scenarios — must work before calling it production-like

1. Employee A tries to GET Employee B's private information → deny; public directory fields still work.
2. Manager M approves team member's pending leave → success + ledger + notification + audit; same manager tries another department → deny.
3. Manager M submits own leave and tries to approve it → deny; delegated approver can handle it.
4. HR corrects leave balance → allowed only via adjustment workflow with reason; employee cannot PATCH balance directly.
5. IT agent queries a payroll/HR ticket by guessing ID → deny including attachments/comments/search.
6. System administrator views account metadata → allowed; tries to retrieve HR case documents without special case grant → deny.
7. Employee removed from support queue keeps an old browser JWT → cannot read queue content after revocation policy takes effect.

8. Two sales people allocate the same chassis during overlapping lease periods → only one transaction succeeds.
9. User cannot edit ticket requester's id, approval decision actor or employee's role via crafted JSON.
10. User uploads a file to one ticket and attempts to download it from another ticket → deny.
11. Employee who changes departments no longer sees previous manager-only resources.
12. Timezone boundary on midnight, holiday overlaps, half-day leave and overnight shift compute correctly under documented policy.
13. Deactivated employee cannot submit new tasks, request leave or clock in, even if an old session remains temporarily valid.
14. Published policy version 2 does not overwrite acknowledgment for version 1; new acknowledgment can be required.
15. All sensitive actions appear in protected audit log with actor, timestamp and event, without storing passwords/secret tokens.

19. References and maintenance notes

Authoritative starting points (review when implementing): - Supabase Auth / user identities: https://supabase.com/docs/guides/auth - Supabase user data model: https://supabase.com/docs/guides/auth/managing-user-data - Supabase Row Level Security: https://supabase.com/docs/guides/database/postgres/row-level-security - Supabase custom claims and RBAC: https://supabase.com/docs/guides/api/custom-claims-and-role-based-access-control-rbac

Scope note: This document is an architecture proposal based on the supplied interface screenshots and user requirements, not a completed security review, formal HR/legal policy or deployed design. Confirm employment and privacy rules with the relevant jurisdiction before real-user deployment. Record every deviation in the ADR table. Source code is unchanged.